WEEK 3 · LESSON 14 OF 24 · HANDOUT

Vector Search and Vector Databases

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=Lha_QCkXxm0 — open the lesson online to watch inside the course.

🎓 **Go deeper:** North Bay Python 2024: A visual exploration of vectors (Pamela Fox) — https://www.youtube.com/watch?v=Q_slEpNuhMc

🎓 **Go deeper:** EuroPython 2024: Fundamentals of Retrieval Augmented Generation — https://www.youtube.com/watch?v=vSwK5sBpD2c

Your goal in this lesson: Build a small vector store in NumPy with search and metadata filters, understand how vector databases scale to millions of items, and choose a practical option.

What a vector store does

A **vector store** keeps three things for every item: the **vector**, the original **text**, and **metadata** (title, date, language, owner…). It answers: "give me the k items whose vectors are closest to this query vector, optionally only those matching a filter".

```
HANDBOOK = {
    "Opening hours": "The Kigali Tech Hub is open Monday to Friday from 8:00 to 20:00 and on
Saturday "
                     "from 9:00 to 14:00. The hub is closed on Sundays and on public
holidays.",
    "Membership plans": "There are three plans. The Student plan costs 10,000 RWF per month and
needs a "
                        "valid student card. The Standard plan costs 25,000 RWF per month. The
Team plan "
                        "costs 90,000 RWF per month for up to five people.",
    "Joining": "To become a member, fill in the online form, bring your national ID or passport
to the "
               "front desk, and pay the first month. Your access card is ready the same day.",
    "Payment": "Fees are paid by mobile money or bank transfer before the 5th of each month.
Members who "
               "pay late for two months lose access until the balance is cleared.",
    "Cancelling": "You may cancel your membership at any time with 14 days' written notice by
email. "
                  "Fees already paid for the current month are not refunded.",
    "Wi-Fi": "The Wi-Fi network is called KTH-Members. The password is printed on your access
card and "
             "changes every three months. Guests use the separate KTH-Guest network.",
    "Meeting rooms": "Members can book the two meeting rooms through the hub app. Standard and
Team members "
                     "get 4 free hours per month; Student members get 1 free hour. Extra hours
cost 5,000 RWF.",
    "Guests": "Each member may bring up to two guests per day. Guests must sign in at the front
desk "
              "and leave by 18:00.",
    "Printing": "Black-and-white printing costs 50 RWF per page and colour printing 200 RWF per
page. "
                "Students get 100 free black-and-white pages per month.",
    "Events": "The hub runs a free coding workshop every Wednesday at 17:00 and a startup pitch
night "
              "on the last Friday of each month. Members can register in the hub app.",
    "Lockers": "Lockers can be rented for 3,000 RWF per month. Lost locker keys cost 5,000 RWF
to replace.",
    "Code of conduct": "Members must respect others, keep noise low in the quiet zone, and
never share their "
                       "access card. Harassment leads to immediate cancellation of
membership.",
    "Lost and found": "Items left behind are kept at the front desk for 30 days, then
donated.",
    "Accessibility": "The ground floor, meeting rooms and toilets are wheelchair accessible. A
lift serves "
                     "all floors. Contact the front desk to arrange other support.",
}
chunks = [{"id": i, "title": t, "text": f"{t}. {body}"} for i, (t, body) in
enumerate(HANDBOOK.items())]
print(len(chunks), "sections")
```

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import Normalizer

texts = [c["text"] for c in chunks]
embedder = make_pipeline(TfidfVectorizer(stop_words="english", sublinear_tf=True),
                         TruncatedSVD(n_components=12, random_state=0),
                         Normalizer())                      # unit length: dot product = cosine
similarity
doc_vectors = embedder.fit_transform(texts)

def embed(text_list):
    return embedder.transform(text_list)

print("each section is now a vector of", doc_vectors.shape[1], "numbers")
```

A vector store in 30 lines

```
class VectorStore:
    def __init__(self):
        self.vectors = None
        self.items = []

    def add(self, texts, metadatas):
        new = embed(texts)
        self.vectors = new if self.vectors is None else np.vstack([self.vectors, new])
        for text, meta in zip(texts, metadatas):
            self.items.append({"text": text, **meta})

    def search(self, query, k=3, where=None):
        scores = self.vectors @ embed([query])[0]          # cosine (vectors are unit length)
        order = np.argsort(scores)[::-1]
        results = []
        for i in order:
            item = self.items[i]
            if where and any(item.get(key) != value for key, value in where.items()):
                continue                                    # metadata filter
            results.append({**item, "score": round(float(scores[i]), 3)})
            if len(results) == k:
                break
        return results

store = VectorStore()
money_topics = {"Membership plans", "Payment", "Cancelling", "Printing", "Lockers"}
store.add([c["text"] for c in chunks],
          [{"title": c["title"], "topic": "money" if c["title"] in money_topics else "general"}
for c in chunks])

for r in store.search("How much does it cost each month?", k=3):
    print(r["score"], r["title"])
print("--- only money topics ---")
for r in store.search("How much does it cost each month?", k=3, where={"topic": "money"}):
    print(r["score"], r["title"])
```

Metadata filters are essential in real apps: search only this customer's documents, only the latest policy, only Kinyarwanda texts.

How fast is brute force?

Comparing the query with every vector is called **exact (brute-force) search**. With NumPy it is surprisingly fast:

```
import time

rng = np.random.default_rng(0)
many = rng.normal(size=(100_000, 384)).astype("float32")      # 100,000 vectors of 384 numbers
many /= np.linalg.norm(many, axis=1, keepdims=True)
query = many[123] + 0.05 * rng.normal(size=384).astype("float32")

start = time.perf_counter()
top5 = np.argsort(many @ query)[::-1][:5]
print(f"searched 100,000 vectors in {(time.perf_counter() - start) * 1000:.0f} ms; best match:
{top5[0]}")
print(f"memory: {many.nbytes / 1e6:.0f} MB")
```

100,000 vectors take well under a second and about 150 MB of memory — enough for many small and medium apps. The best match is item **123**, the one we disguised with noise.

Scaling up: approximate nearest neighbours

For millions of vectors, comparing with everything becomes too slow. **Approximate nearest-neighbour (ANN)** indexes (such as HNSW graphs or clustered indexes) check only a small, promising part of the data, returning almost the same results in milliseconds — trading a tiny bit of accuracy for big speed-ups.

Choosing a vector store

Option	Good for
NumPy (like ours)	Learning, up to ~100,000 items
FAISS (library)	Fast search inside one Python program
Chroma, LanceDB	Simple embedded vector databases for prototypes
pgvector (PostgreSQL extension)	Apps that already use PostgreSQL — vectors next to your normal tables
Qdrant, Weaviate, Milvus, Pinecone	Large dedicated vector databases and managed services

If your app already uses PostgreSQL (like the WAMUGII platform), **pgvector** is often the simplest choice: one database, normal backups, and SQL filters with vector search.

Keep vectors in sync

When a document changes, re-embed it and replace its vector; when it is deleted, delete its vector. Store which embedding model produced each vector — vectors from different models cannot be compared.

Key terms

- **Vector store / vector database** — stores vectors with text and metadata and finds the nearest ones.
- **Metadata filter** — restricts search to items with certain properties.
- **Exact (brute-force) search** — compare with every vector.
- **Approximate nearest neighbours (ANN)** — fast indexes that check only part of the data.
- **pgvector** — vector search inside PostgreSQL.

Remember this: A vector store is vectors plus text plus metadata with a nearest-neighbour search; brute force is fine for small collections, ANN indexes and vector databases handle millions.

Name: Date:

Activity

Task 1

Add a `delete(title)` method to `VectorStore`.

Task 2

Add a "language" field to the metadata and store two sections translated into Kinyarwanda. Filter by language.

Task 3

Time the brute-force search with 10,000, 100,000 and 300,000 vectors. Is the growth linear?

Task 4

Read the pgvector documentation and write the SQL to create a table with a 384-number vector column.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does a vector store keep for each item?
   - A. The vector, the original text and metadata
   - B. The API key
   - C. Only the vector
   - D. Only the text
2. What is a metadata filter used for?
   - A. Changing the embedding model
   - B. Searching only items with certain properties, such as one customer's documents
   - C. Making vectors longer
   - D. Deleting the store
3. What is brute-force (exact) search?
   - A. Searching by keywords
   - B. Guessing
   - C. Comparing the query with every vector
   - D. Searching only the first item
4. Why use an approximate nearest-neighbour index?
   - A. To encrypt data
   - B. To make embeddings
   - C. To get perfect results
   - D. To search millions of vectors quickly by checking only part of the data
5. Your app already uses PostgreSQL. Which option is often simplest?
   - A. A spreadsheet
   - B. Printing vectors
   - C. pgvector
   - D. A second app
6. Why record which embedding model made each vector?
   - A. To speed up search
   - B. Vectors from different models cannot be compared
   - C. It is not needed
   - D. For billing

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.